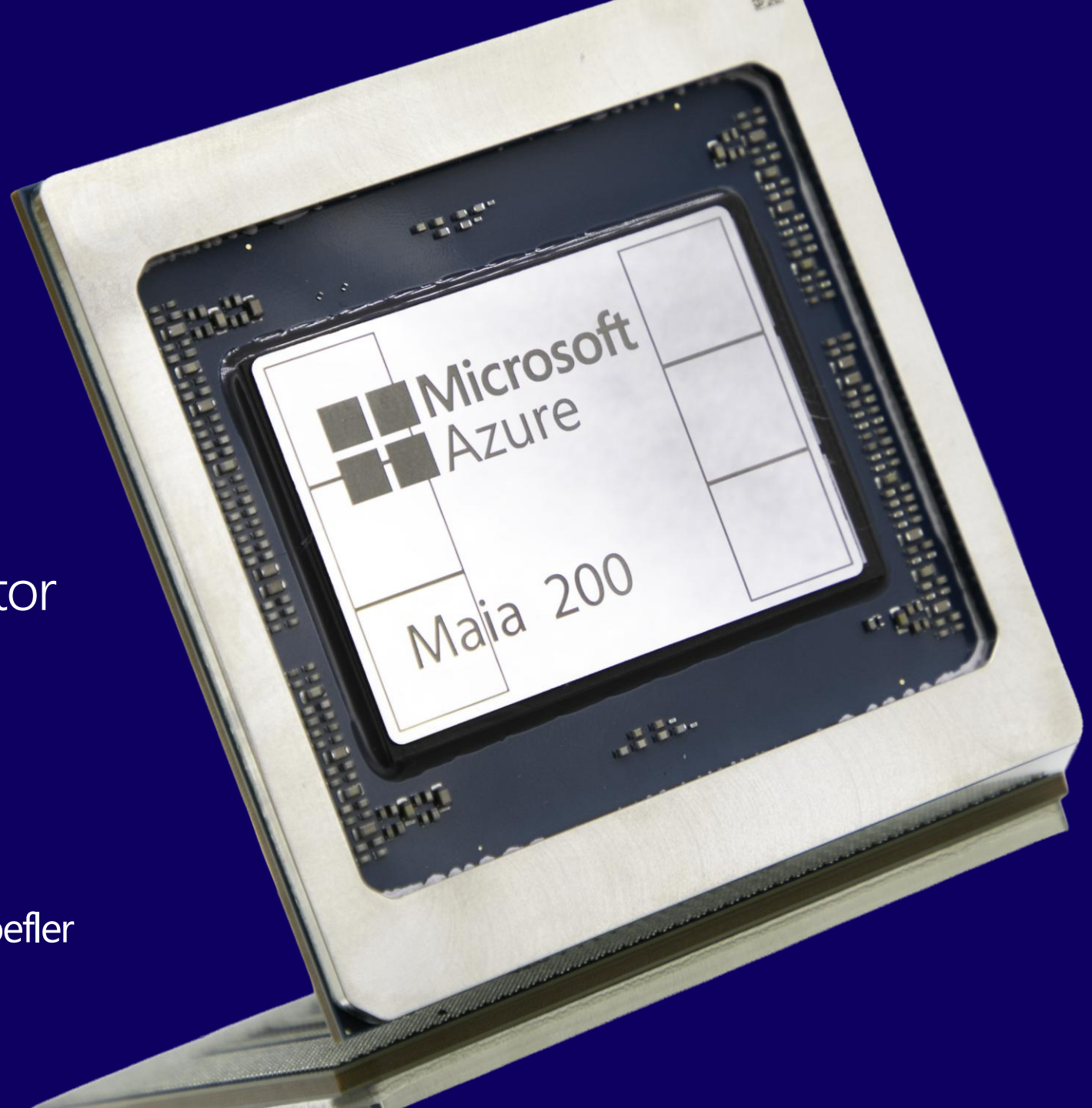
Microsoft

Maia 200

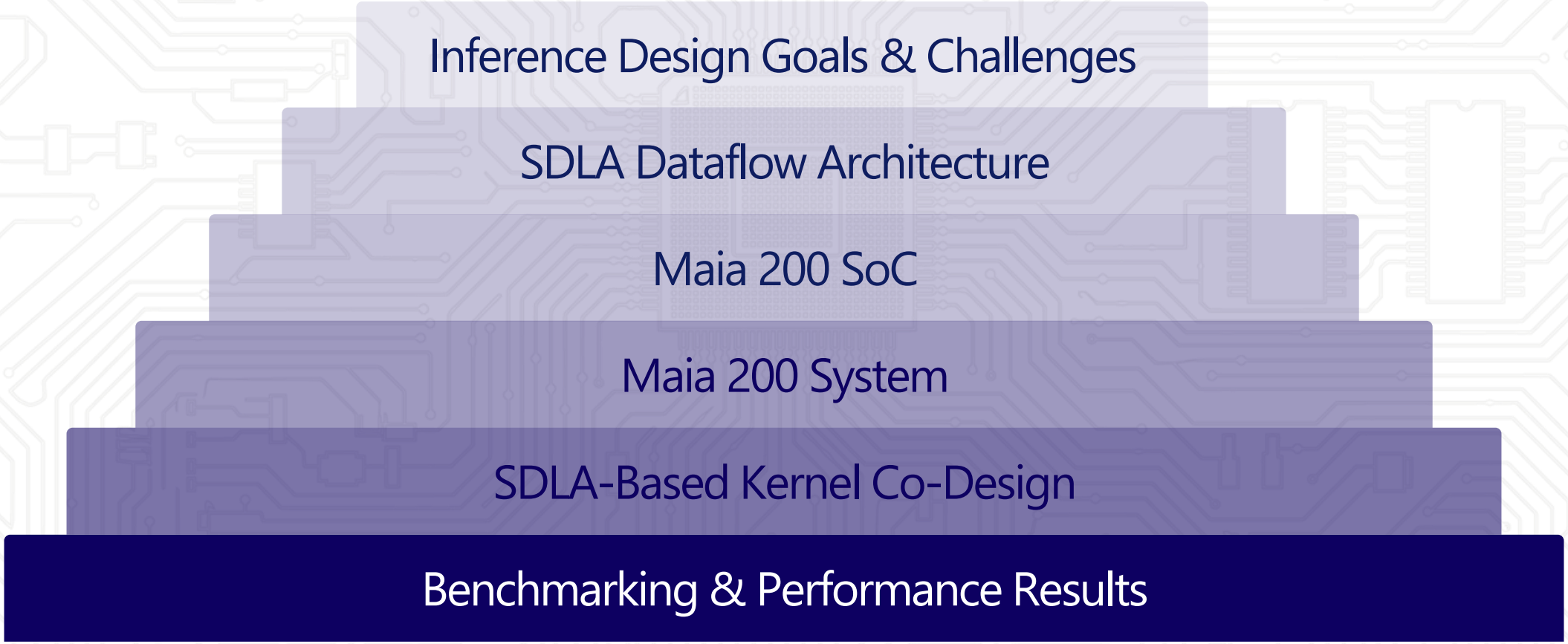
A Data Center Scale AI Accelerator for Large Scale Inference using Software Defined Dataflow



Hot Chips 2026

Prashant Ranjan, Jackson Peng, Torsten Hoefler

Co-designing Maia 200
The Most Efficient Inference Engine on Azure



Inference Design Goals & Challenges

Inference Design Goals & Challenges

Diverse Workloads

- Prefill vs Decode
- Mixture of Experts
- Agentic Interactions
- Programmable

Diverse Applications

- Interactive Chat
- Summarization
- Deep Reasoning
- Deep Research

Challenging SLAs

- Low Latency Tokens
- High Throughput
- High Availability
- High Reliability

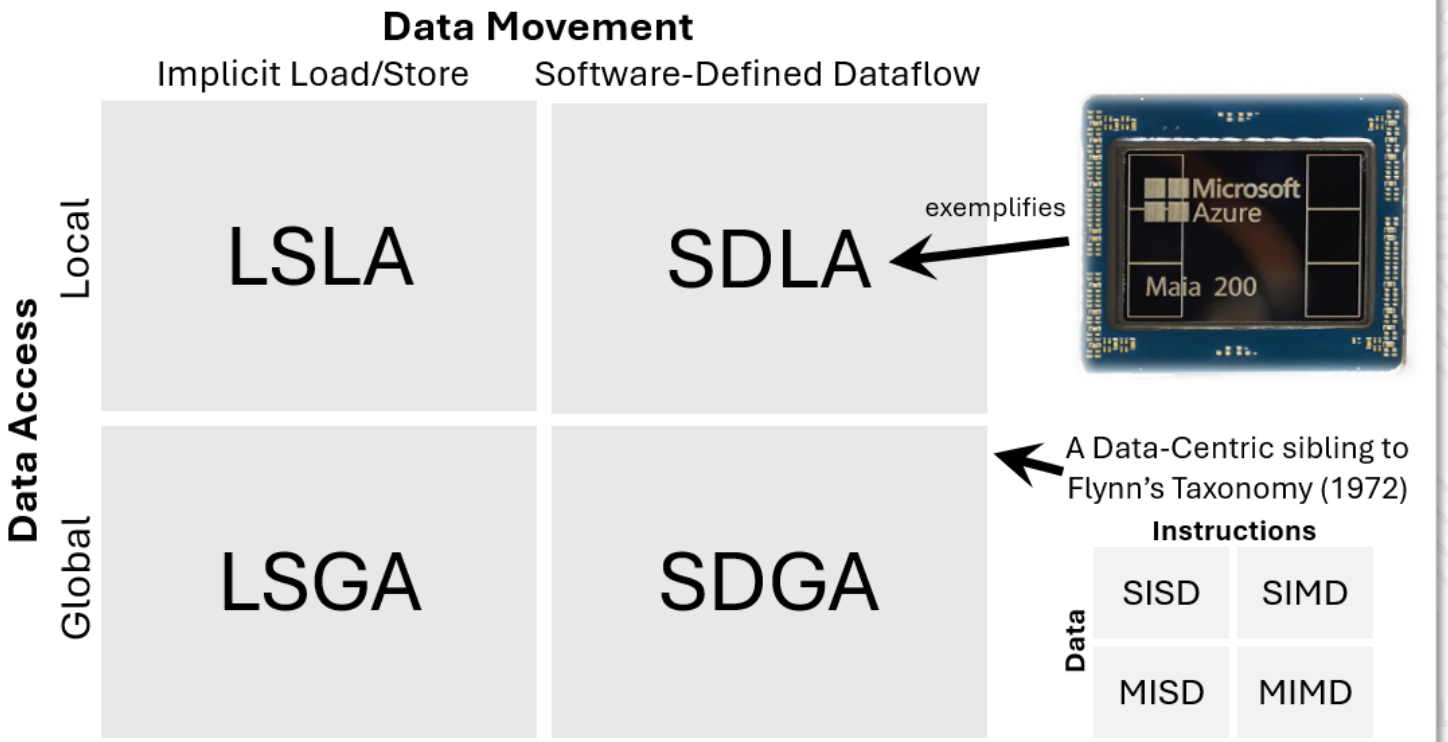
Designing Infrastructure to Deliver all this at: Minimum $/Token & W/Token

SDLA Dataflow Architecture

Software Defined Local Access Dataflow

SDLA Dataflow Architecture

Software Defined Local Access Dataflow



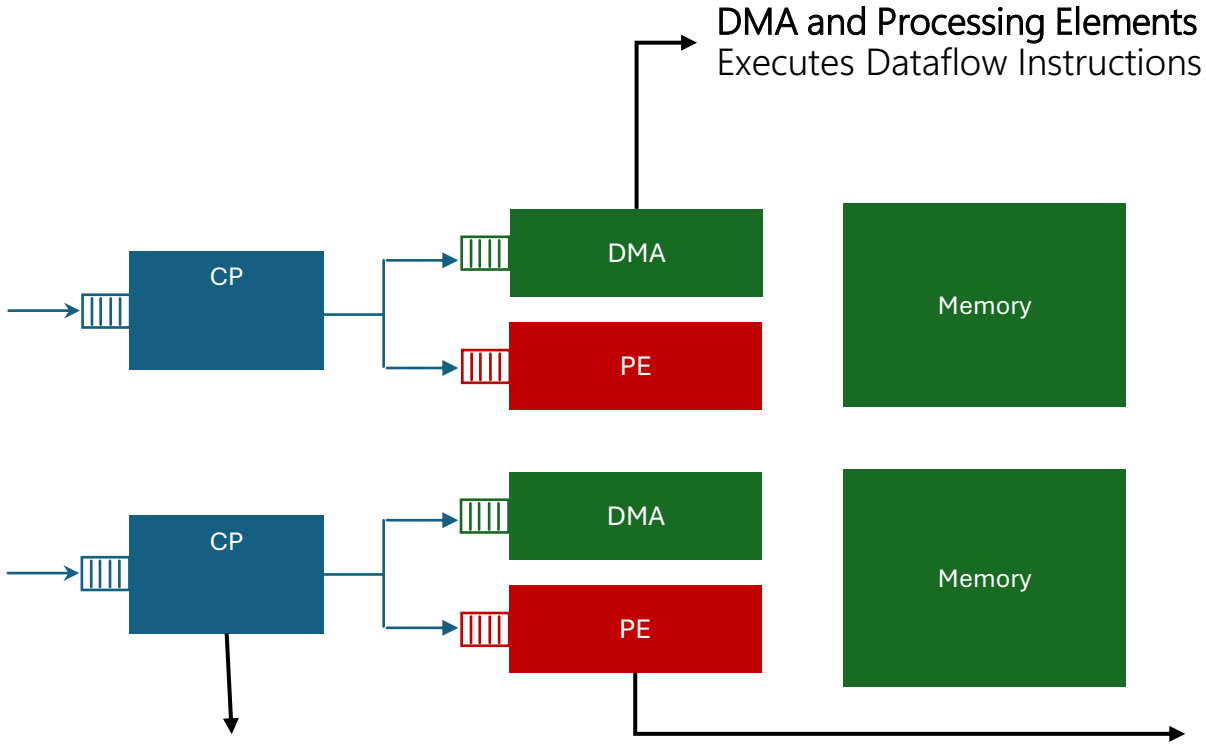
SDLA forms a New Class of Architecture

- Explicit software orchestration
- Independent control and data streams
- Data obliviousness
- Locally accessed

Maia 200 SoC

Control | Compute | I/O | Semaphores

Asynchronous Control & Data Path



DMA and Processing Elements (PEs) run the data path:
Executes Dataflow Instructions

Dataflow Instruction

Pre0	Pre1	Pre2	Instruction	Post0	Post1	Post2

Precondition on up to 3 semaphores (wait)

- DMA command
- kernel invocation
- Network tx / rx

Postcondition on up to 3 semaphores (signal)

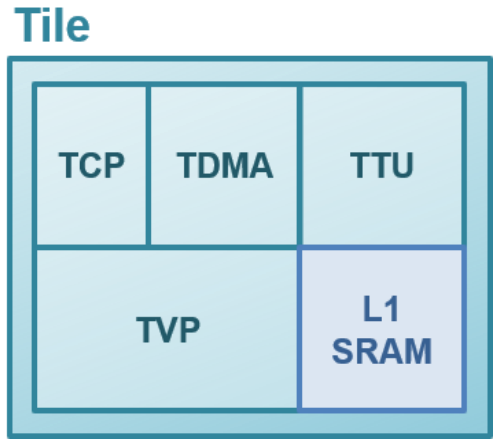
Control Processors (CPs) run the control path:
Enqueues Dataflow Instructions to the accelerator queues

HW Queues with Macro-Instructions:

- In-order execution of Macro-Instructions
- Explicit synchronization through Semaphores between HW queues

High Throughput Compute

Each Tile Packages Compute, Local Memory &
Data Movement Control into a Self-Contained Unit



Tile Tensor Unit (TTU)

- High-throughput matrix multiply and convolution engine
- Optimized for low precision block scaled data types (FP8/FP6/FP4)
- Support Hardware data type conversion at line rate through TDMA

Tile Vector Processor (TVP)

- Fully programmable SIMD engine
- Maintains flexibility for model and algorithm changes
- Support Hardware data type conversion at line rate through TDMA

Tile Control Processor (TCP)

- Executes control code to orchestrate the SDLA dataflow
- Coordinates TTU, TVP, and DMA tasks
- Controls hardware semaphores for fine-grained sync between movement & execution

Hierarchical Data Storage & Movement

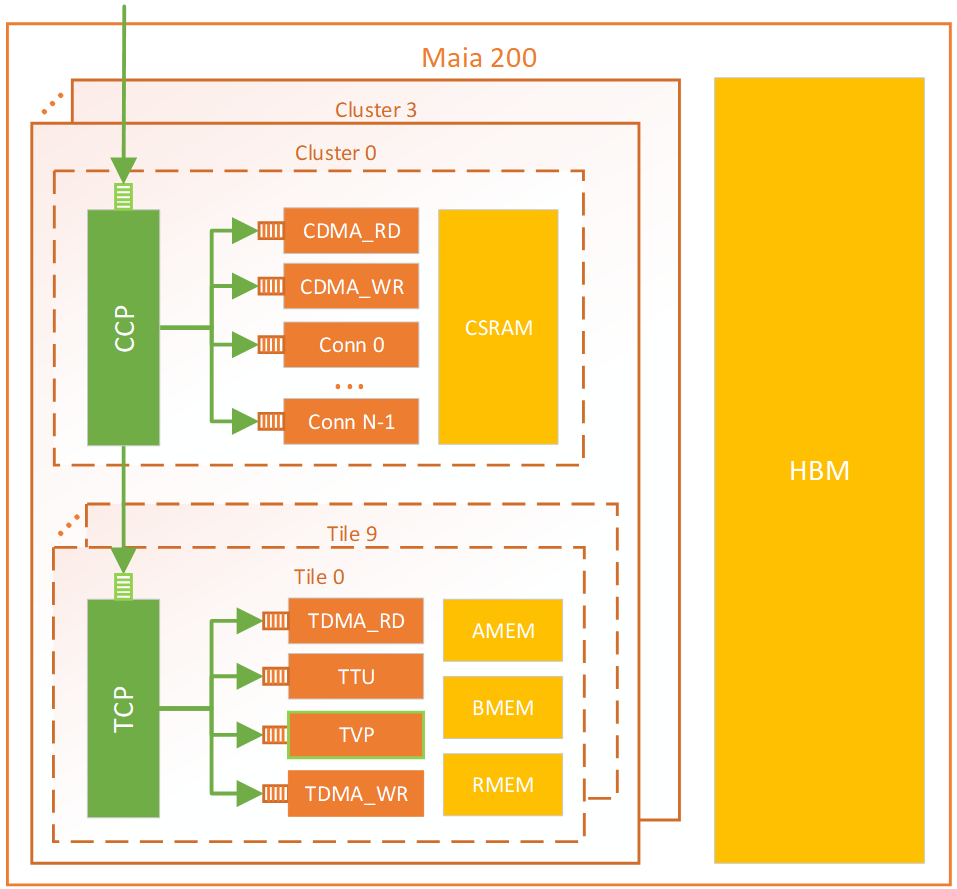
Local Data Access

Hierarchical Data Storage

- L1 (TSRAM) and L2 (CSRAM) to facilitate data re-use and reduce data movement

Hierarchical Data Movement

- Hierarchy of DMA engines
- Tile (TSRAM) <-> Cluster (CSRAM) <-> Chip (HBM)
- Cluster to Cluster Broadcast



I/O: Custom NIC & Interconnect Protocol

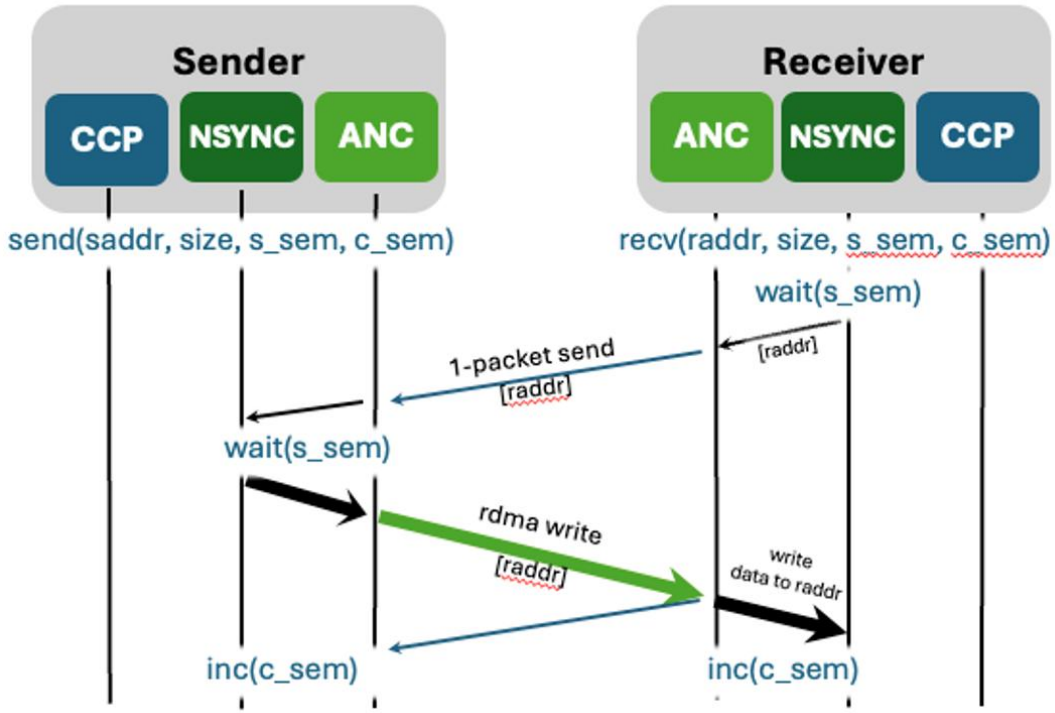
Higher Performance at Low Power & Area

Custom Embedded AI NIC (ANC)

Custom Scale Up Fabric Transport (ATL)

- Ethernet based
- End point controlled Multi Pathing support (Packet Spray, load balancing, OOO Receiver)
- Supports multi-tier network
- Reliability focused (Hardware based Fast failure detection and Recovery)
- End2End encrypted

Low Power (<8pJ/b), Latency (<1us, P2P, one-way mem2mem) and Size



ATL had key influence on standardization of the Ultra Ethernet Transport (UET) and Multipath Reliable Connection (MRC) Specifications

Load Balancing: Intra & Inter Chip

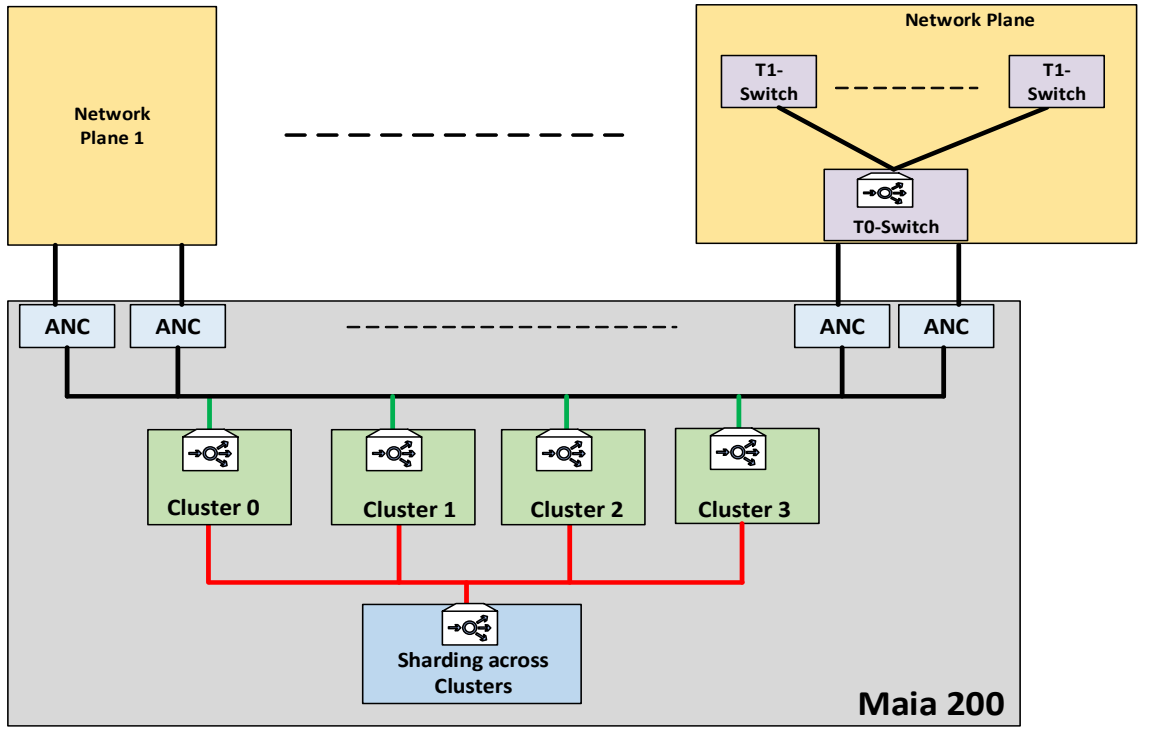
Load balancing at every level is fundamental to scale performance with the system

Intra-Chip Load Balancing

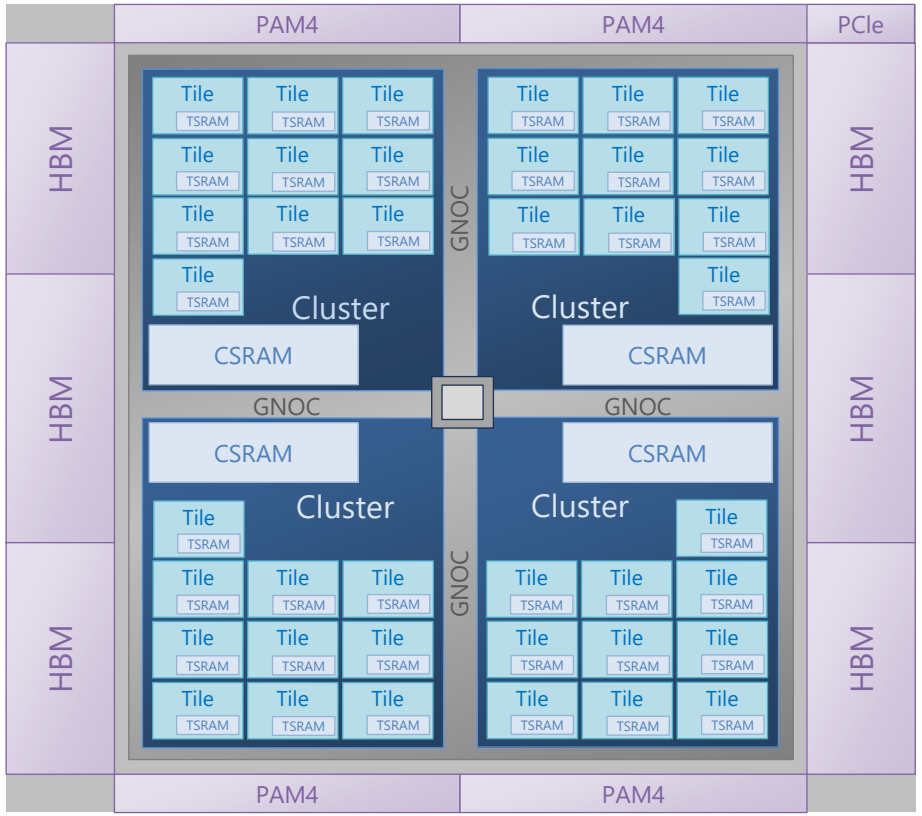
- Hierarchical NOC and Compute
- Intra-Chip Multicast/Broadcast support
- Unified HBM

Inter-Chip Load Balancing

- Sharding across ANCs
- FCQ (Fully connected Quad)
- Load balancing in Network



Putting It All Together : Maia 200 SOC



	Maia 200
Peak Dense Tensor TOPS	FP4: 10,145 FP8: 5,072 BF16: 1,268
Peak HBM BW (TB/s)	7 TB/s
Number of HBM Stacks	6
HBM Capacity (GB)	216
SRAM Capacity (MB)	272
SRAM BW (TB/s)	80
Host PCIe BW (GB/s)	64
PCIe Configuration	Gen6 x8
Backend Network Bandwidth (GB/s, Uni-directional)	1,400
Backend Network Configuration	28x400
SOC Die Area (mm2)	~820
Package Size	75x75
SoC TDP (provision)	750W
Process Technology	TSMC 3nm
Package Technology	CoWoS-S

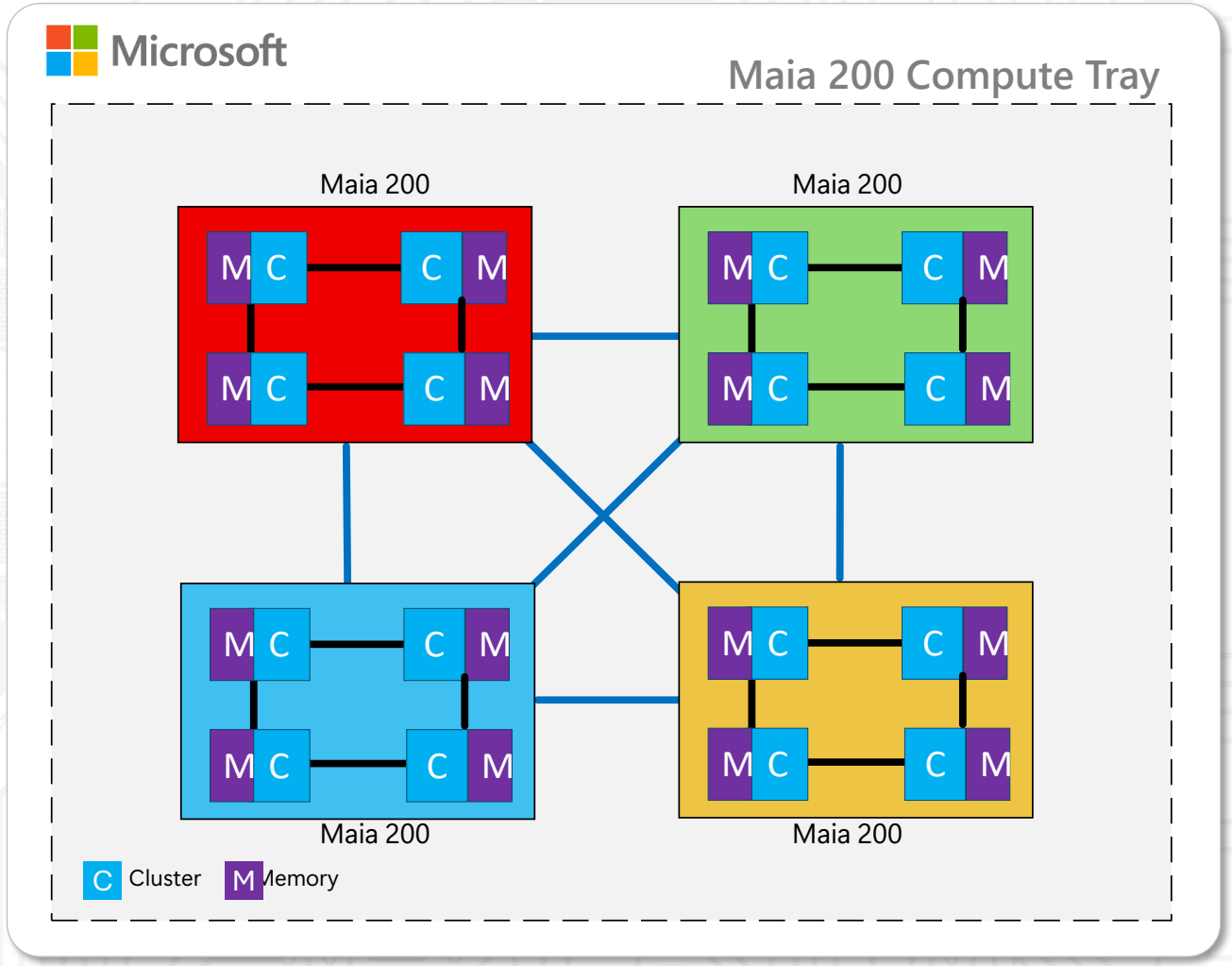
Maia 200 System

From Chips to Cluster

FCQ Topology: Fully Connected Quad

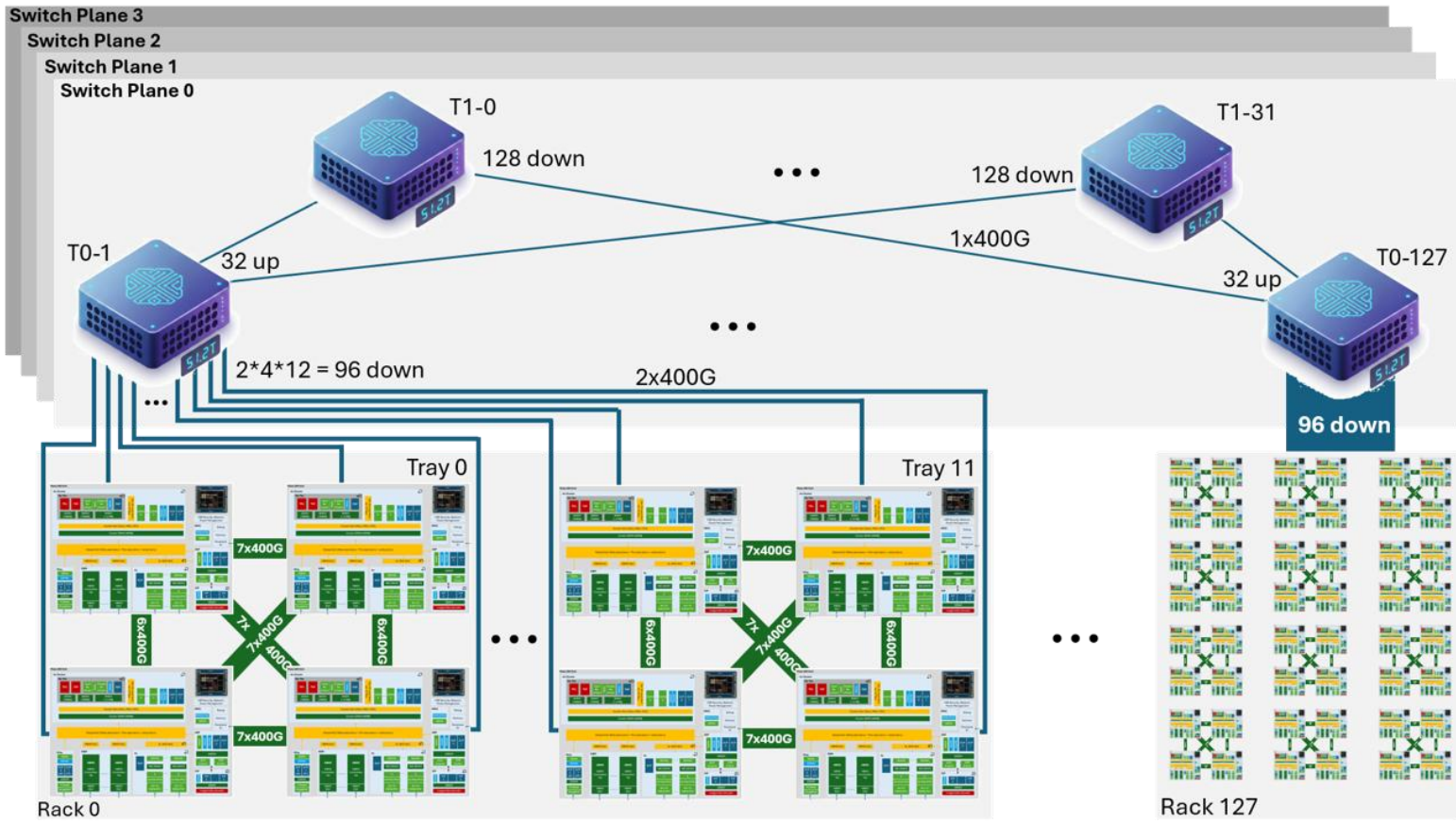
Maximizing local bandwidth for tensor parallelism

- Tray Level: Fully Connected Quad (FCQ)
- 4 Accelerators directly connected via fixed Ethernet links (no switch)
- Maximum bandwidth between nearest neighbors for TP-heavy operations
- Efficient AllGather and AllReduce within the Quad



Unified Ethernet Scale-Up Network

One Network, One Protocol, from Chip to 6k Accelerator Cluster



Why Unified?

- Single Ethernet-based fabric
- Same ATL stack everywhere
- Standard switch ecosystem

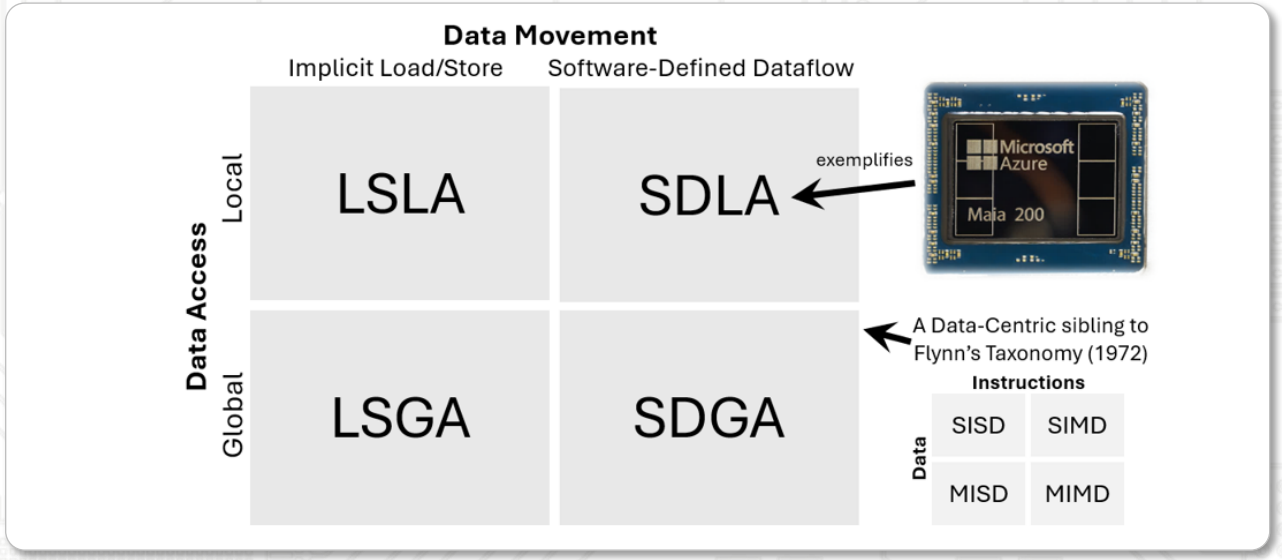
Optimized Maia 200 I/O Stack

Communication	Microsoft Collective Communication Library (MCCL)	• Efficient Non-blocking MPI implementation (minimize pipeline bubbles) • Hides Synchronization Overhead (pipelined commands) • Simple software APIs • Algorithmic Optimizations (reduce incast, maximize overlap)
Compute and IO Distribution	E2E Load Balanced	• Distributed Workload Parallelization Strategy • Intra Chip Sharding • IO Load Balancing Throughout (SOC to System)
Network	Fixed (FCQ) + Switched (Two Tier Unified Clos)	• RDMA based • Transport offload • E2E Congestion Control • Reliability focused
Connectivity	Custom Interconnect	• Integrated NIC (ANC) • Custom Transport Protocol (ATL) • Area, Power and Latency Optimized

SDLA-Based Kernel Co-Design

Compute & Collective Kernel Co-Design & Optimization

Kernel Co-Design



SDLA-Driven Design

- Fully asynchronous programming model
- Separate data flow, data computation and control flow; and achieve fully-overlapped operations
- Efficiently utilizing a localized hierarchical memory subsystem

SDLA Based Kernel Implementation Allows HW/SW Codesign

- Let HW/SW orchestrate the best data movement and placement at all development phase
- A predictable data movement overhead at early phase of silicon development
- Optimized SoC NoC implementation for best P(ower)-P(erformance)-A(rea)

Kernel Design (Batch GEMM)

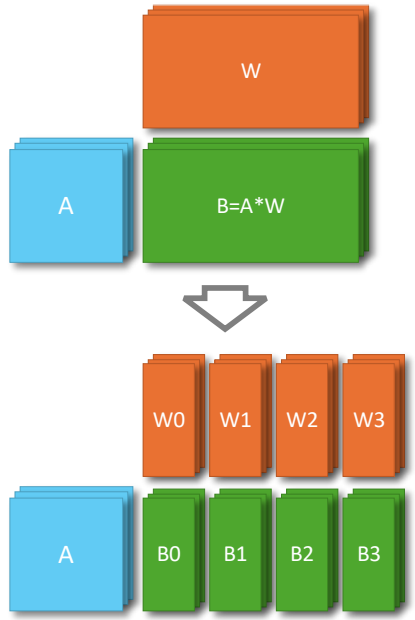
Gather Based Implementation

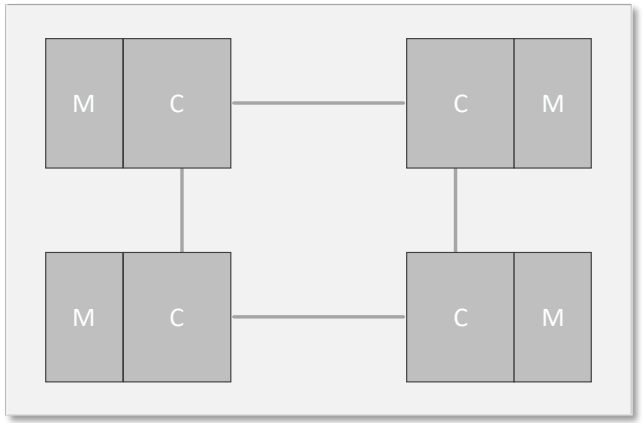
On-chip Data Flows

- Activation: gather (broadcast) over central GNOC
- Weight: load from nearby HBM
- Output: pinned in the tile L1 SRAM

Benefit

- Fully utilize the broadcast NOC to send low-precision activation while pinning high-precision outputs
- Reduce the weight movement for power saving.





Kernel Design (Batch GEMM)

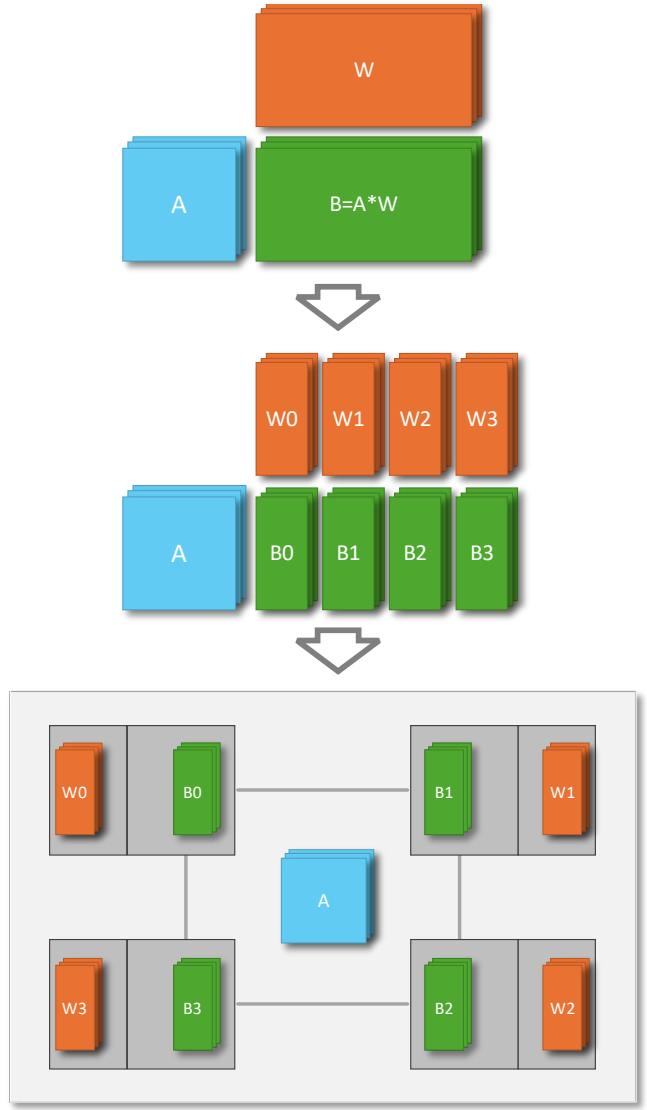
Gather Based Implementation

On-chip Data Flows

- Activation: gather (broadcast) over central GNOC
- Weight: load from nearby HBM
- Output: pinned in the tile L1 SRAM

Benefit

- Fully utilize the broadcast NOC to send low-precision activation while pinning high-precision outputs
- Reduce the weight movement for power saving.



Kernel Design (Batch GEMM)

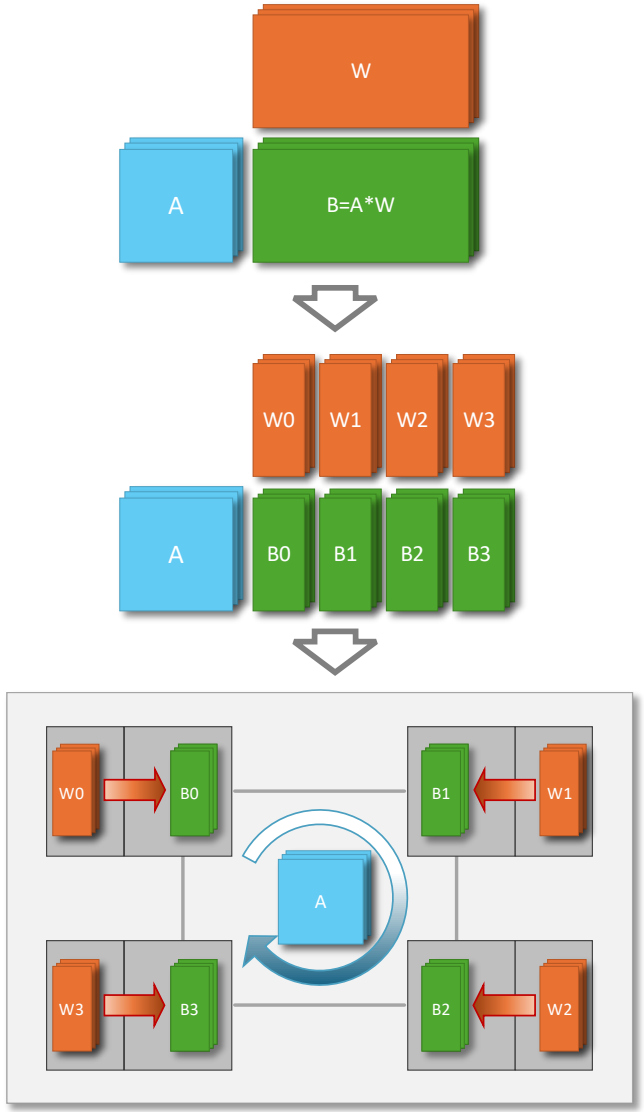
Gather Based Implementation

On-chip Data Flows

- Activation: gather (broadcast) over central GNOC
- Weight: load from nearby HBM
- Output: pinned in the tile L1 SRAM

Benefit

- Fully utilize the broadcast NOC to send low-precision activation while pinning high-precision outputs
- Reduce the weight movement for power saving.



Kernel Design Distributed (Batch GEMM)

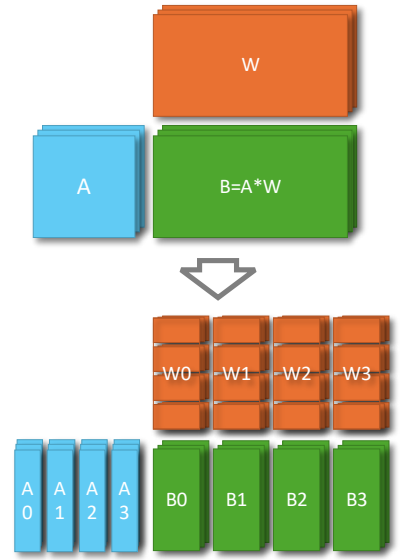
Gather-Based Implementation

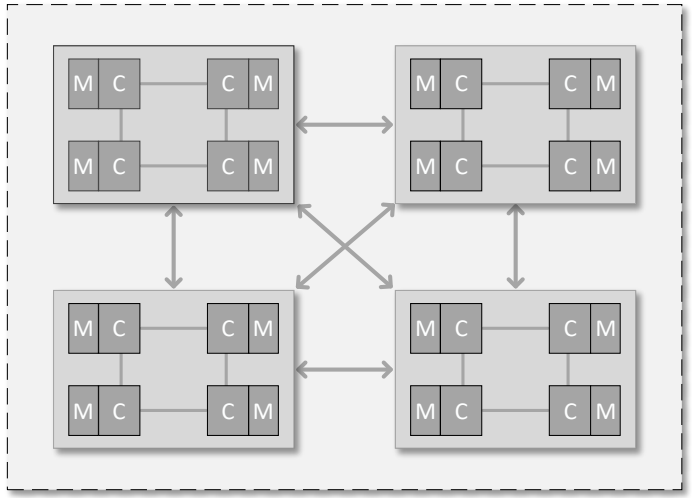
System level data flows

- Activation: gather (broadcast) over network
- Each chip run (Batch) GEMM on each chunk of received activations

Benefit

- Overlap collective and GEMM at fine granularity





Kernel Design Distributed (Batch GEMM)

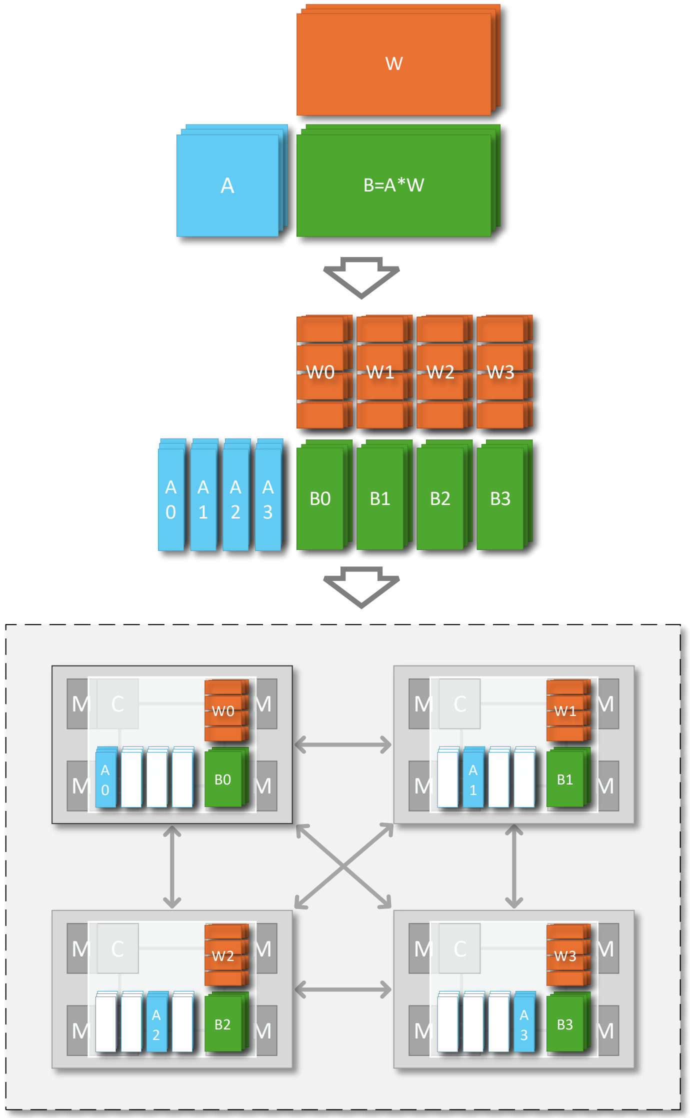
Gather-Based Implementation

System level data flows

- Activation: gather (broadcast) over network
- Each chip run (Batch) GEMM on each chunk of received activations

Benefit

- Overlap collective and GEMM at fine granularity



Kernel Design Distributed (Batch GEMM)

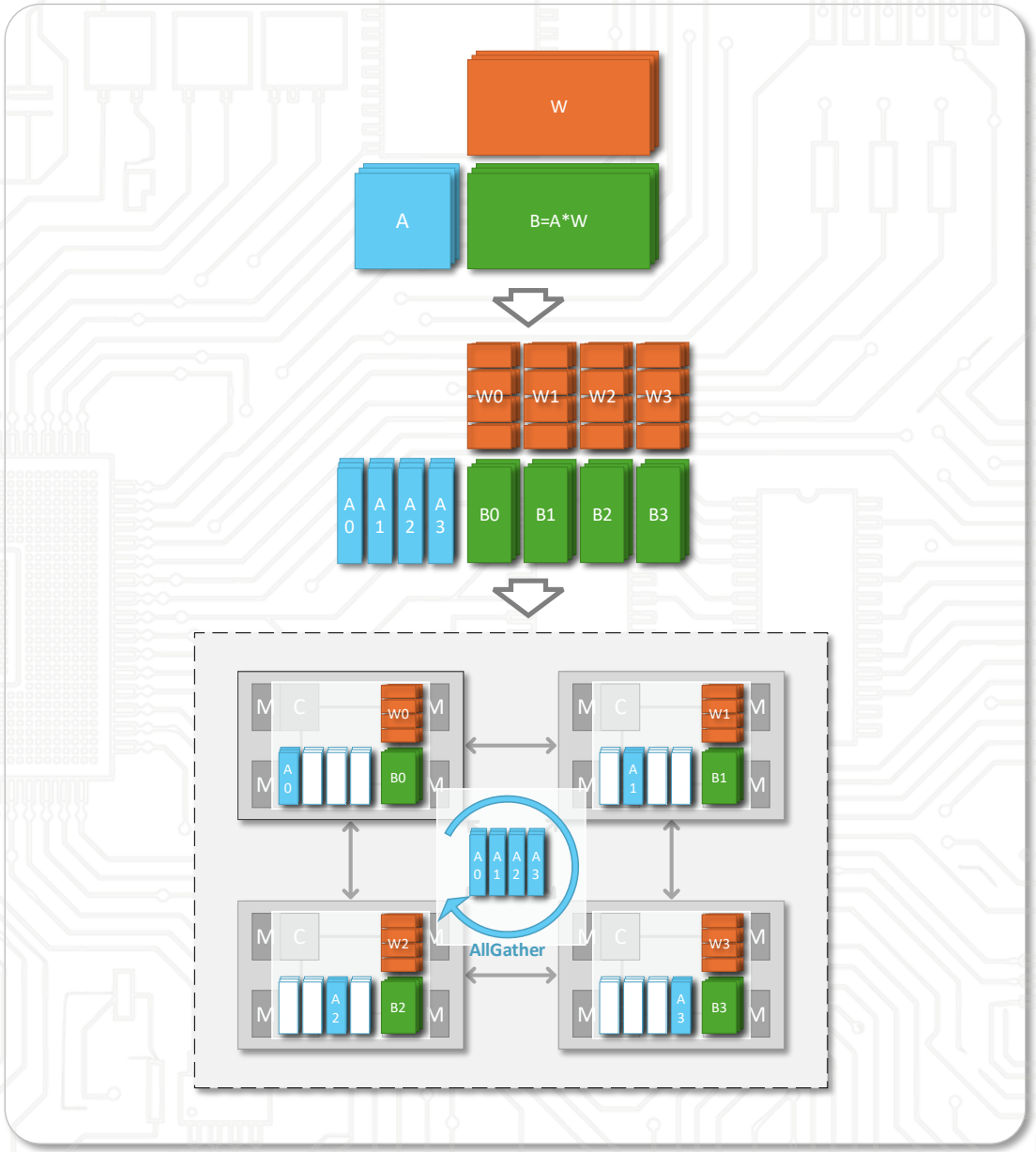
Gather-Based Implementation

System level data flows

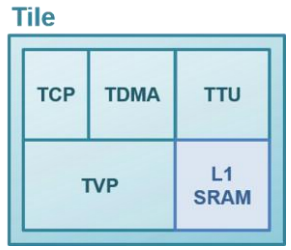
- Activation: gather (broadcast) over network
- Each chip run (Batch) GEMM on each chunk of received activations

Benefit

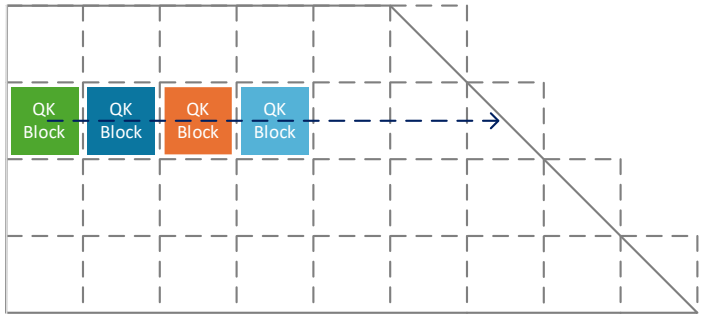
- Overlap collective and GEMM at fine granularity



Kernel Design Attention (Scaled-Dot-Product)



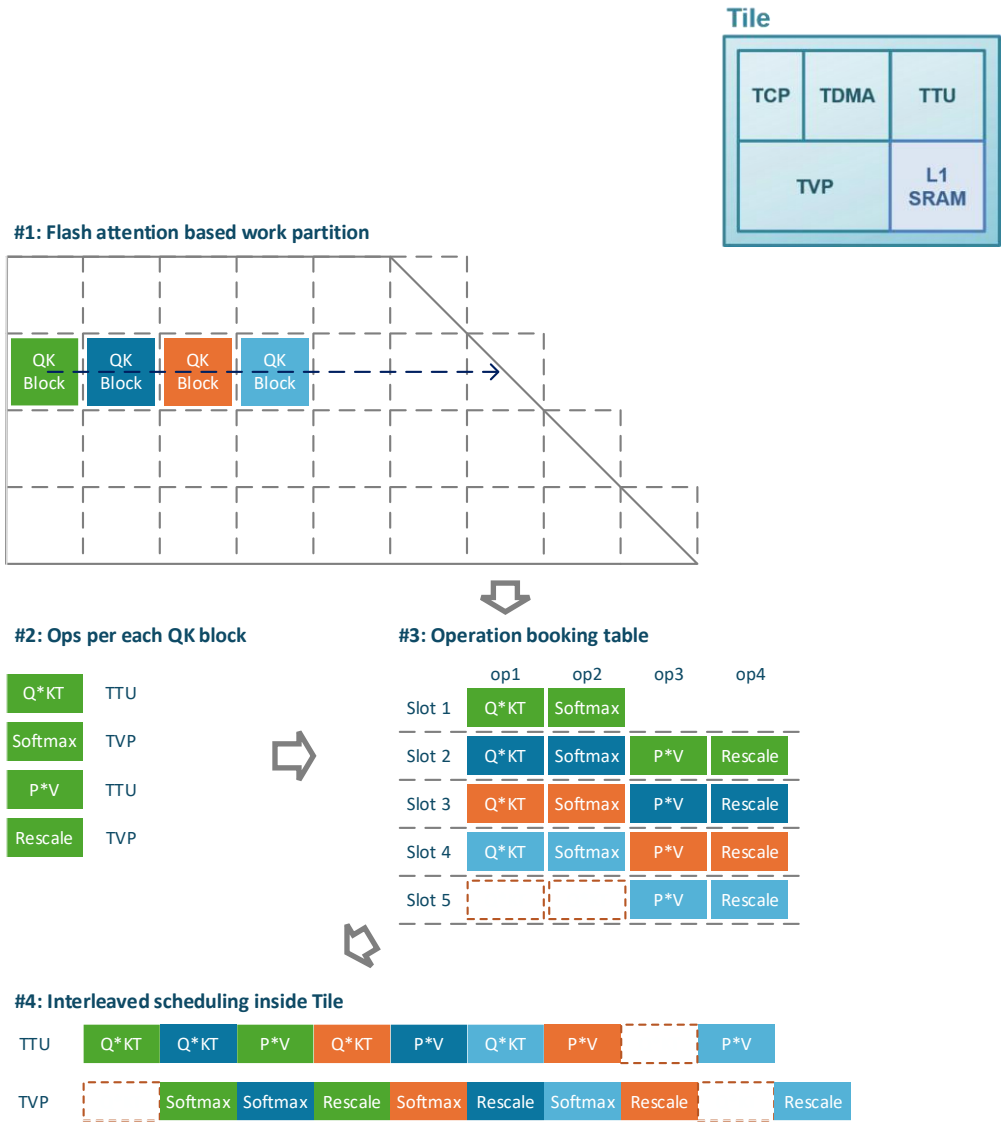
#1: Flash attention based work partition



- Fully adopt to flash attention variants and can either pin Q(uery)/O(utput) or pin K(ey)/V(alue) in TSRAM
- Block interleaving to get fully overlapped 2 Tensor operations and 2 SIMD operations

Kernel Design Attention (Scaled-Dot-Product)

- Fully adopt to flash attention variants and can either pin Q(uery)/O(utput) or pin K(ey)/V(alue) in TSRAM
- Block interleaving to get fully overlapped 2 Tensor operations and 2 SIMD operations



Kernel Design Collectives

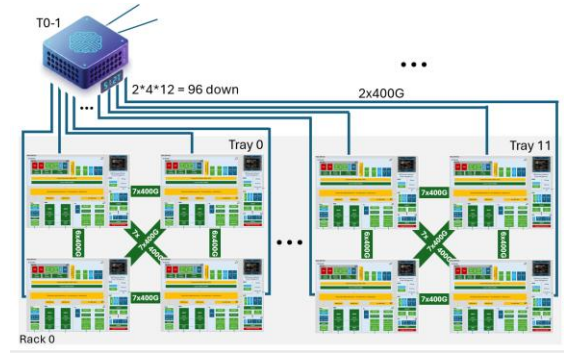
An adaptive logical topology

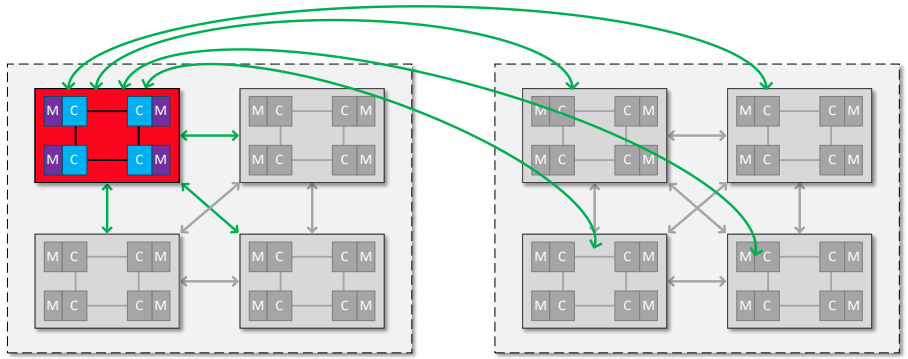
Broadcast Based implementation

- Fit for small sized transfer for low latency

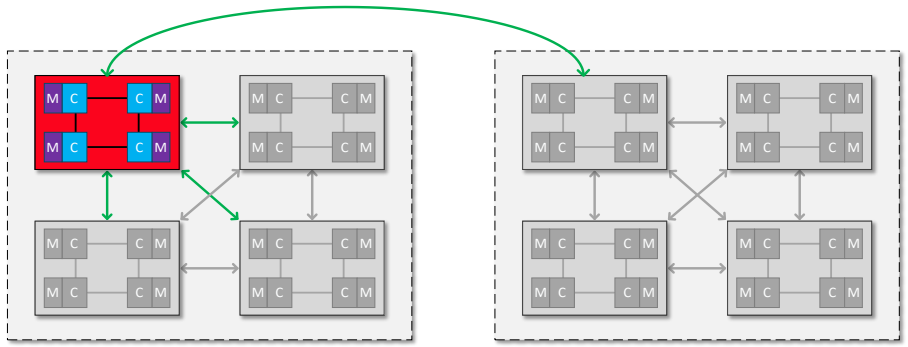
Hierarchical Based Implementation

- Fit for medium sized transfer for balanced throughput/latency





Broadcast Based Collectives



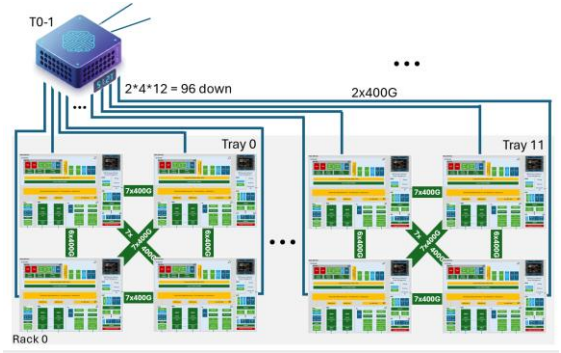
Hierarchical Based Collectives.

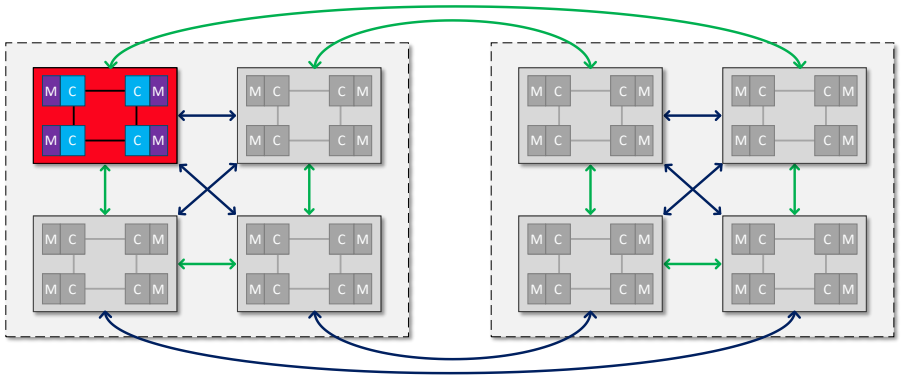
Kernel Design Collectives

An adaptive logical topology

Ring Based Implementation

- Fit for large sized transfer for peak throughput





Ring Based Collectives

Performance Results

Kernel Performance (as of June 2026)

GEMM

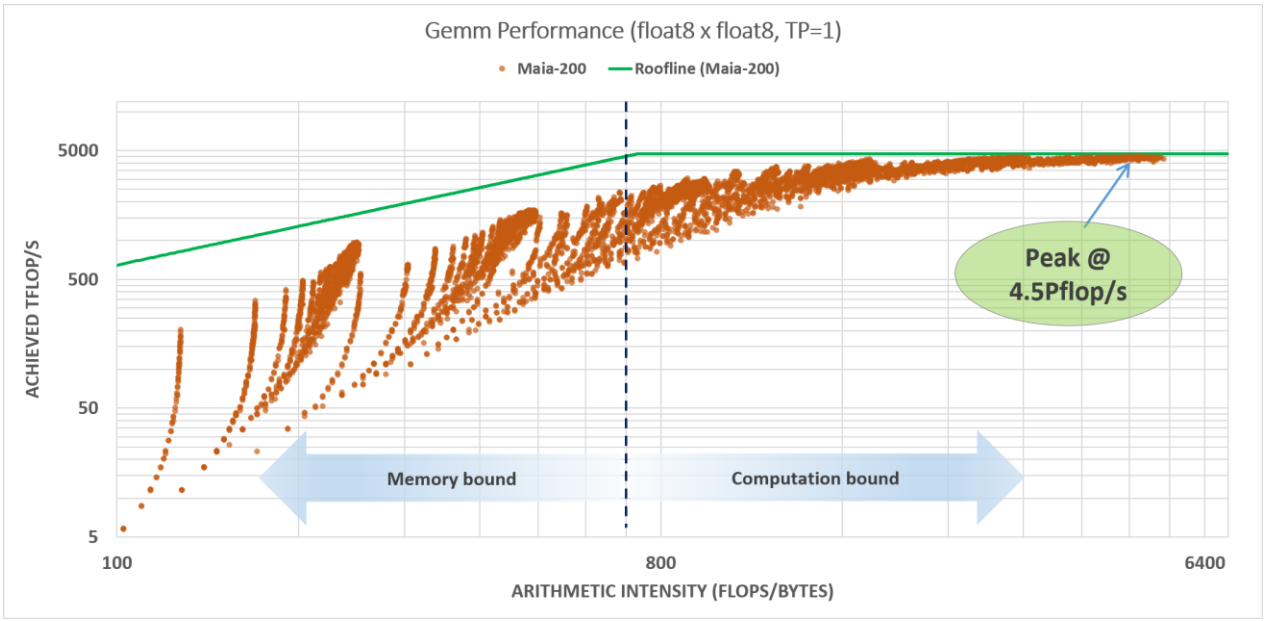
Benchmark

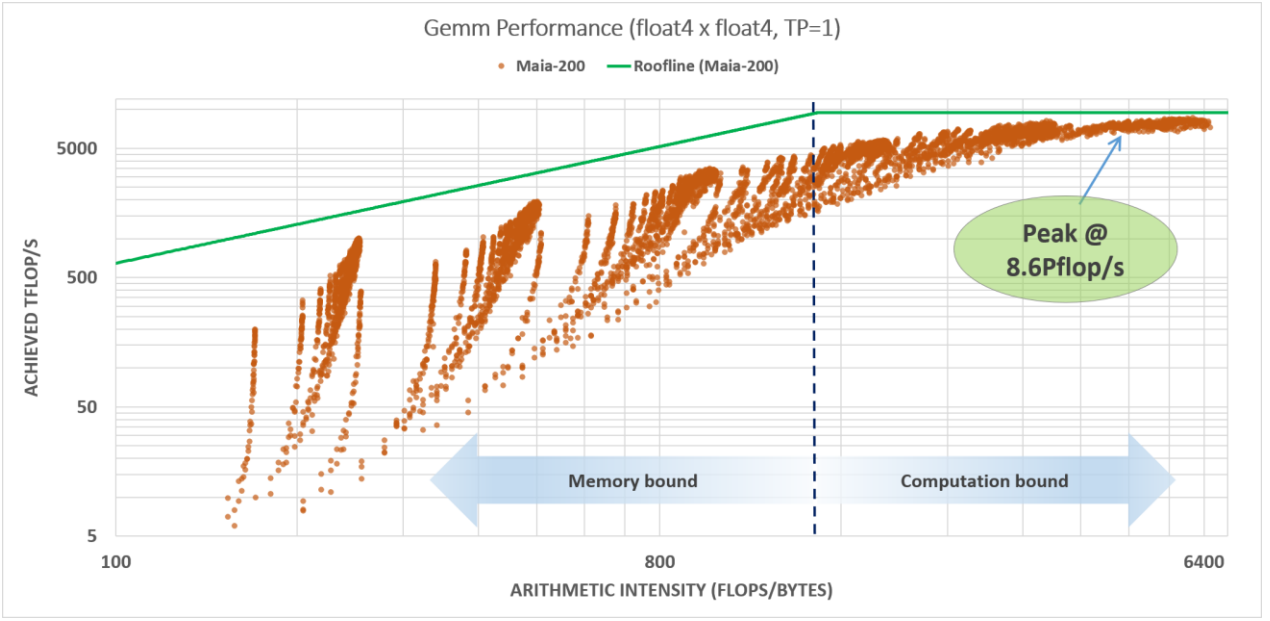
- Float8 * Float8, TP=1
- Float4 * Float4, TP=1

Throughput – Compute Intensity Chart

- Roofline vs Achieved

$$Arithmetic\ Intensity = \frac{Tensor\ Ops}{I/O}$$





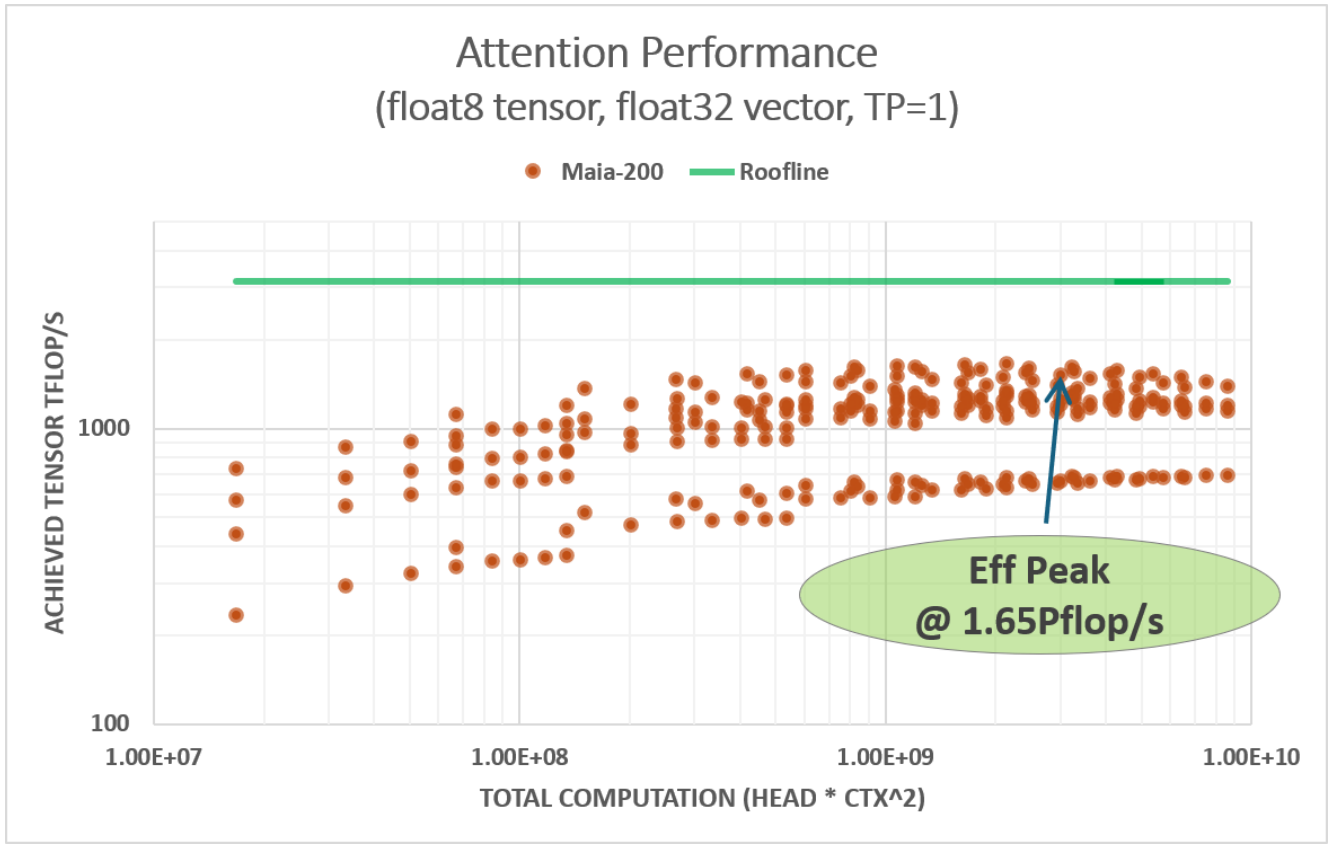
Attention

Benchmark

- Fused scaled-dot-product w/ flash attention 2 implementation
- Tensor: Float8, SIMD: Float32
- GQA = 4

Throughput Chart

- Roofline vs Achieved
- Computation := QHead * Context ^ 2



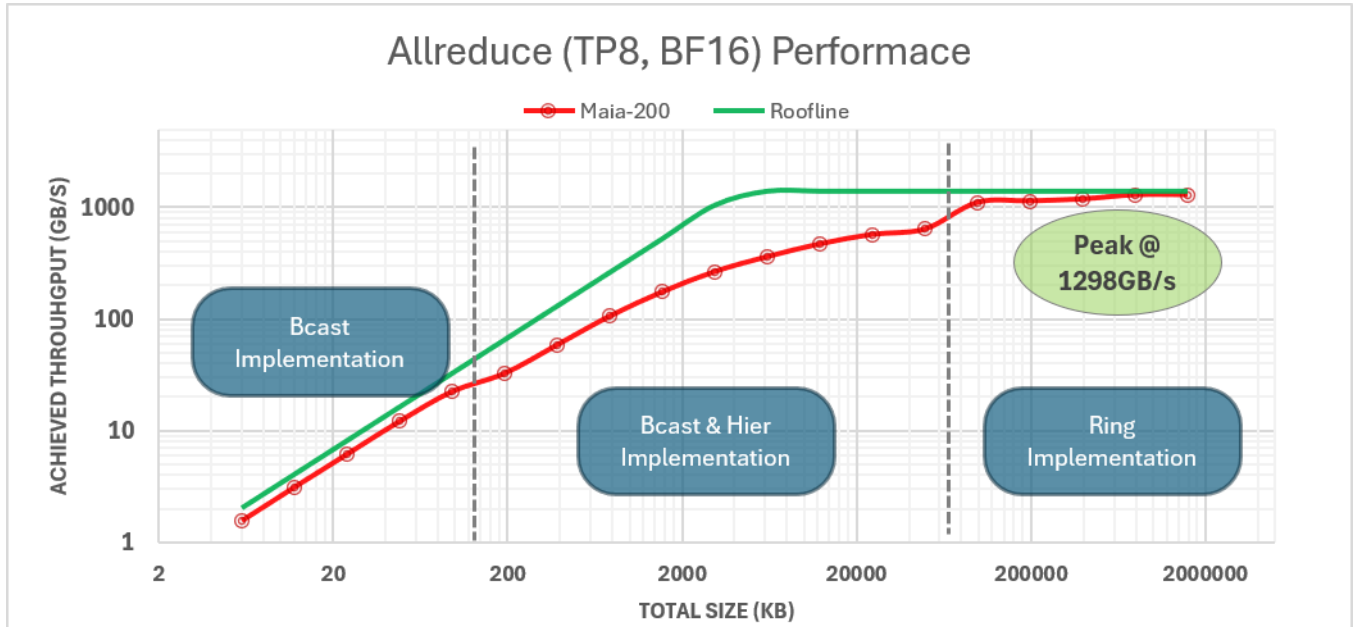
Collective

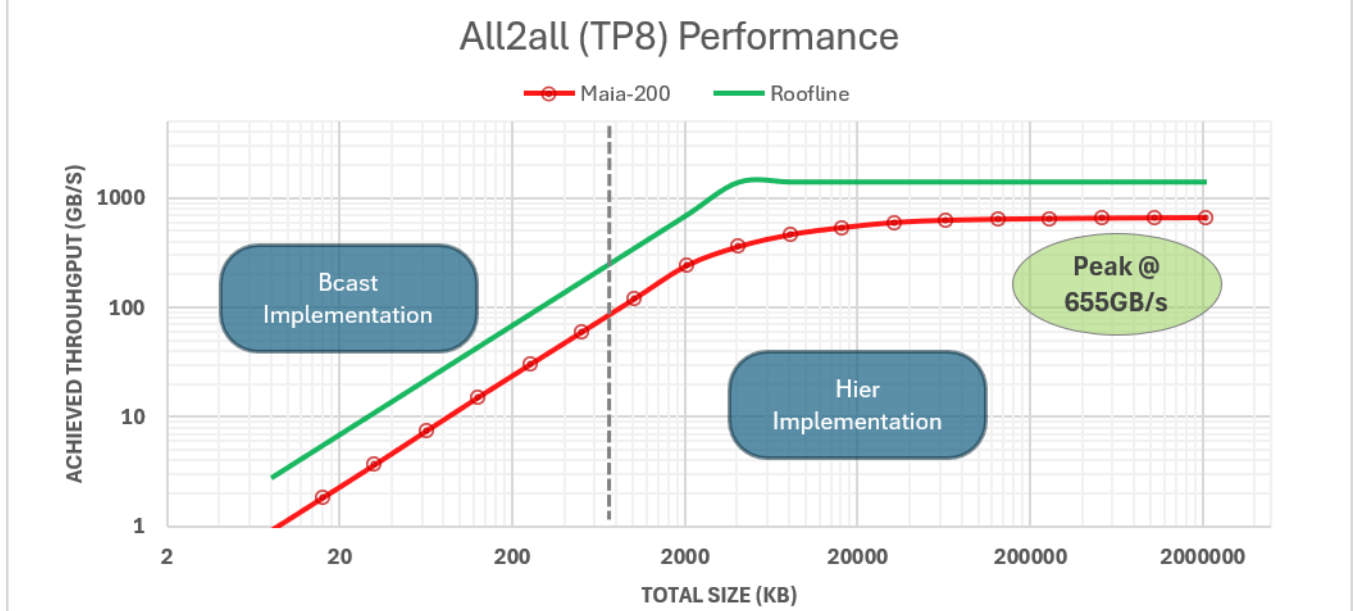
Benchmark

- Allreduce (BF16), TP8
- Alltoall, TP8

Throughput – Transfer Size Chart

- Roofline vs Achieved





Summary & Key Takeaways

Vertically Co-Designed SoC & System

TTU, DMA hierarchy, memory tiers, on chip NOC, integrated NICs and Software SDKs

SDLA Architecture

A new class of data-movement-centric architectures

Optimized System Design

Unified All Ethernet Network

Kernel Co-Design

Optimized compute + communication kernels design

Result : Lower TCO, Better Energy Efficiency

Higher Performance /$

Microsoft

Q&A